

High-Level Synthesis of Hardware Accelerators using Vitis

Processor Design
3rd Report

Alba Soldevilla González

May 26, 2026



UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH

Contents

1	Project Topic and Objectives	2
2	Tools	2
2.1	Vitis & Vivado workflow	2
3	GPU4S Bench	3
3.1	LRN_bench	3
3.2	cifar_10	4
3.3	cifar_10_multiple	5
3.4	convolution_2D_bench	6
3.5	correlation_2D	7
3.6	fast_fourier_transform_2D_bench	8
3.7	fast_fourier_transform_bench	10
3.8	fast_fourier_transform_window_bench	11
3.9	finite_impulse_response_filter	12
3.10	matrix_multiplication_bench	13
3.11	matrix_multiplication_bench_fp16	14
3.12	matrix_multiplication_tensor_bench	14
3.13	max_pooling_bench	15
3.14	memory_bandwidth_bench	16
3.15	relu_bench	17
3.16	softmax_bench	18
3.17	wavelet_transform	19
4	Results Summary & Discussion	22
4.1	Comparison with Bambu	22
4.2	Description of the results	24
5	Implementation Effort and HLS Constraints	26
5.1	Effort Analysis	26
5.2	Critical Assessment	26
6	Conclusions	27
7	Future Work	27
A	FPGA Synthesis Metrics	28
B	Excluded Datatype Configurations	29

1 Project Topic and Objectives

For this project, I have been assigned the task of converting software into hardware designs using **High-Level Synthesis (HLS)** techniques and different hardware design tools. The main goal is to translate C/C++ code into synthesizable Hardware Description Language (HDL) implementations such as VHDL and Verilog.

Once the conversion is completed, the generated hardware designs will be synthesized for **Field Programmable Gate Arrays (FPGAs)**. The resulting implementations will then be evaluated using a set of benchmarks in order to analyze their efficiency.

The overall workflow of the project will be as follows:

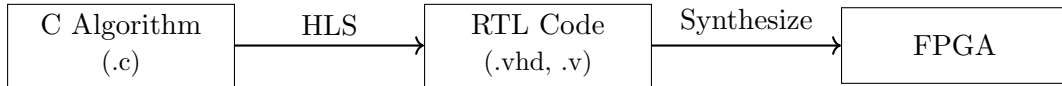


Figure 1: Simplified workflow of the hardware generation process.

2 Tools

For this report, we extended the project by exploring an alternative HLS tool: **AMD Vitis HLS**. Vitis is part of the AMD/Xilinx unified development platform and provides a high-level synthesis flow, enabling the generation of RTL hardware descriptions directly from C/C++ code.

The generated hardware designs are then synthesized and implemented using **AMD Vivado**, a design suite for AMD FPGAs that covers the full implementation flow: synthesis, placement, routing, and verification.

The workflow presented in Figure 1 can be updated as shown in Figure 2.

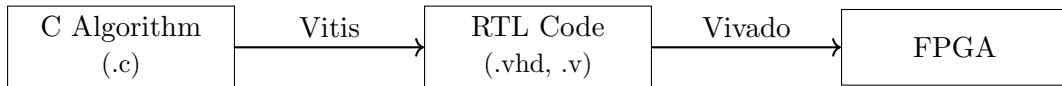


Figure 2: Toolchain used in this project.

All experiments were carried out on Kubuntu 24.04 LTS (Intel Core i7-1365U) using **AMD Vitis HLS 2025.2** and **Vivado v2025.2**, targeting the xc7z020c1g484-1 (Zynq-7020) device.

2.1 Vitis & Vivado workflow

The equivalent workflow using AMD tools is divided into two main stages. First, the hardware description is generated from C/C++ using **Vitis HLS**. Then, the resulting RTL is synthesized and implemented on the target FPGA using **Vivado**.

The HLS stage is executed through a **Vitis HLS script**, `run_hls.tcl`:

```
1 $ vitis_hls -f run_hls.tcl
```

The `run_hls.tcl` script defines the HLS project configuration, including the top-level function, the input source files, the target FPGA device, and the clock constraint.

Once the RTL has been generated, it is passed to Vivado for implementation using a **Vivado script**, `run_vivado.tcl`:

```
1 $ vivado -mode batch -source run_vivado.tcl
```

The `run_vivado.tcl` script automates the backend FPGA flow. It creates a Vivado project targeting the same device, imports the RTL generated by Vitis HLS, and sources the IP generation scripts required to correctly instantiate complex components such as floating-point operators. Each benchmark has its adapted `run_hls.tcl` and `run_vivado.tcl`.

Additionally, the script generates detailed reports to evaluate the performance and efficiency of the generated hardware design.

3 GPU4S_Bench

The GPU4S_Bench suite is an open-source collection of benchmarks covering computational kernels representative of space applications, including signal processing, neural network operations, and linear algebra. In this section, the HLS workflow is applied to each benchmark to obtain resource utilization, timing, and power consumption metrics. A description of each reported metric is provided in Appendix A.

Note on source modifications. For benchmarks requiring source modifications, a copy of the original file was created under the name `cpu_functions_vitis.cpp`.

Note on I/O utilization. Several benchmarks report a logical I/O count exceeding the 200 physical pins available on the `xc7z020clg484-1` device. This occurs because Vitis HLS generates one top-level port per array element when array-based interfaces are used, resulting in a large number of logical signals. Since only post-synthesis results are reported and full implementation (placement and routing) was not performed, no error is raised.

Note on mathematical IP cores. Several kernels use transcendental or non-trivial arithmetic operations such as `exp()`, `pow()`, `sqrt()`, `cosf()`, and `sinf()`. Vitis HLS synthesizes these into dedicated floating-point IP cores rather than simple combinational logic. To ensure correct synthesis, all `*_ip.tcl` scripts generated by Vitis HLS are sourced in the Vivado script before reading the RTL, and their targets are generated prior to synthesis. This pattern is applied silently to all affected benchmarks.

3.1 LRN_bench

This benchmark implements the Local Response Normalization (LRN) function, commonly used in neural network inference. The HLS synthesis target is the `lrn` function.

Compilation Issues and Fixes

No issues were reported beyond the standard mathematical IP generation described in the preamble of Section 3.

Scripts used

Vitis HLS script:

```
1 open_project proj_lrn
2 set_top lrn
3 add_files cpu_functions.cpp -cflags "-DFLOAT"
4 open_solution "solution"
5 set_part xc7z020clg484-1
```

```

6 create_clock -period 9.66
7 csynth_design

```

Vivado script:

```

1 create_project vivado_lrn ./vivado_lrn -part xc7z020clg484-1 -force
2 file mkdir vivado_reports
3 foreach ip_tcl [glob -nocomplain ./proj_lrn/solution/syn/verilog/*_ip.
   tcl] {
4     source $ip_tcl
5 }
6 read_verilog [glob ./proj_lrn/solution/syn/verilog/*.v]
7 generate_target all [get_ips]
8 synth_design -top lrn -part xc7z020clg484-1
9 create_clock -period 9.66 -name default [get_ports ap_clk]
10 report_utilization -file vivado_reports/utilization_synth.rpt
11 report_timing_summary -file vivado_reports/timing_synth.rpt
12 exit

```

Synthesis Results

A summary of the resource utilization and timing results is shown in Table 1.

Resource	Used	Available	Utilization (%)
LUT	1662	53200	3.12
LUTRAM	0	17400	0.00
FF	1750	106400	1.64
BRAM	10.5	140	7.50
DSP	103	220	46.82
I/O	103	200	51.50
Frequency (MHz)	103.5, period = 9.66 ns		
WNS	5.776 ns		

Table 1: Vivado results for `lrn` (Zynq-7020).

3.2 cifar_10

This benchmark implements a full CIFAR-10 inference pipeline, consisting of two convolutional blocks followed by two fully connected layers and a softmax output. The HLS synthesis target is the `cifar10` function, which internally calls several sub-functions such as `lrn` and `softmax`.

Compilation Issues and Fixes

- **I/O overutilization during placement:** The HLS-generated design exposes wide AXI interfaces (e.g., `m_axi_gmem`), translating into hundreds of top-level ports that exceed the physical pin count of the device when full implementation is attempted.
- **Solution adopted:** The design was synthesized in Vivado’s *out-of-context* (OOC) mode, avoiding the mapping of AXI interfaces to physical FPGA pins and allowing correct resource and timing estimation for the accelerator logic itself.

Scripts used

Vitis HLS script:

```

1 open_project proj_cifar10
2 set_top cifar10
3 add_files cpu_functions.cpp -cflags "-DFLOAT"
4 open_solution "solution"
5 set_part xc7z020clg484-1
6 create_clock -period 10.0 -name default
7 csynth_design
8 exit

```

Vivado script:

```

1 create_project vivado_cifar10 ./vivado_cifar10 -part xc7z020clg484-1 -
  force
2 file mkdir vivado_reports
3 foreach ip_tcl [glob -nocomplain ./proj_cifar10/solution/syn/verilog/*
  _ip.tcl] {
4     source $ip_tcl
5 }
6 read_verilog [glob ./proj_cifar10/solution/syn/verilog/*.v]
7 generate_target all [get_ips]
8 synth_design -top cifar10 -part xc7z020clg484-1 -mode out_of_context
9 create_clock -period 10.0 -name default [get_ports ap_clk]
10 report_utilization -file vivado_reports/utilization_synth.rpt
11 report_timing_summary -file vivado_reports/timing_synth.rpt
12 exit

```

Synthesis Results

A summary of the resource utilization and timing results is shown in Table 2.

Resource	Used	Available	Utilization (%)
LUT	21424	53200	40.27
LUTRAM	1879	17400	10.80
FF	30349	106400	28.52
BRAM	11	140	7.86
DSP	136	220	61.82
I/O	242	200	121.00
Frequency (MHz)	100.0, period = 10 ns		
WNS	-1.092 ns		

Table 2: Vivado OOC synthesis results for cifar10 (Zynq-7020).

3.3 cifar_10_multiple

The `cifar_10_multiple` benchmark extends `cifar_10` by processing multiple input images within a single function call via an outer loop, while keeping the internal CNN computation unchanged. The HLS synthesis target is the `cifar10` function.

Compilation Issues and Fixes

The same I/O overutilization issue described in Section 3.2 applies here and was addressed in the same way (OOC synthesis mode).

Scripts used

The same scripts as in Section 3.2 were used, with `proj_cifar10` replaced by the corresponding project directory for this benchmark. No other modifications were required.

Synthesis Results

A summary of the resource utilization and timing results is shown in Table 3.

Resource	Used	Available	Utilization (%)
LUT	26662	53200	50.12
LUTRAM	1746	17400	10.03
FF	33767	106400	31.74
BRAM	10.5	140	7.50
DSP	160	220	72.73
I/O	844	200	422.00
Frequency (MHz)	100.0, period = 10 ns		
WNS	1.393 ns		

Table 3: Vivado OOC synthesis results for `cifar10_multiple` (Zynq-7020).

3.4 convolution_2D_bench

The `convolution_2D_bench` benchmark computes a 2D convolution between an input matrix and a kernel, applying zero-padding at the borders. The HLS synthesis target is the `matrix_convolution` function.

Compilation Issues and Fixes

No issues beyond excessive logical I/O ports (see Section 3 preamble). Full implementation was skipped; only post-synthesis results are reported.

Scripts used

Vitis HLS script

```
1 open_project matrix_convolution_hls
2 set_top matrix_convolution
3 add_files cpu_functions.cpp -cflags "-DFLOAT"
4 open_solution "solution1"
5 set_part xc7z020clg484-1
6 create_clock -period 10.0 -name default
7 csynth_design
8 exit
```

For this benchmark, other flags such as `-DINT` and `-DDOUBLE` can also be used.

Vivado script:

```
1 create_project vivado_matrix_convolution ./vivado_matrix_convolution -
  part xc7z020clg484-1 -force
2 file mkdir vivado_reports
3 foreach ip_tcl [glob -nocomplain ./matrix_convolution_hls/solution1/syn/
  verilog/*_ip.tcl] {
4     source $ip_tcl
5 }
```

```

6 read_verilog [glob ./matrix_convolution_hls/solution1/syn/verilog/*.v]
7 synth_design -top matrix_convolution -part xc7z020clg484-1
8 create_clock -period 10.0 -name default [get_ports ap_clk]
9 report_utilization -file vivado_reports/utilization_synth.rpt
10 report_timing_summary -file vivado_reports/timing_synth.rpt
11 exit

```

Synthesis Results

A summary of the resource utilization and timing results is shown in Table 4.

Resource	Available	-DINT		-DFLOAT		-DDOUBLE	
		Used	Util. (%)	Used	Util. (%)	Used	Util. (%)
LUT	53200	1587	2.98	1669	3.14	2078	3.91
LUTRAM	17400	0	0.00	0	0.00	30	0.17
FF	106400	2588	2.43	2956	2.78	3874	3.64
BRAM	140	0	0.00	0	0.00	0	0.00
DSP	220	39	17.73	41	18.64	50	22.73
I/O	200	167	83.50	167	83.50	263	131.50
Frequency (MHz)	—	100.0 (10 ns)		100.0 (10 ns)		100.0 (10 ns)	
WNS (ns)	—	1.590		1.590		1.590	

Table 4: Vivado results for `matrix_convolution` (Zynq-7020).

3.5 correlation_2D

This benchmark computes the correlation coefficient between two matrices. The HLS target is the `correlation_2D` function, which calculates mean values and performs the final normalized accumulation.

Compilation Issues and Fixes

No issues beyond the standard mathematical IP generation described in the preamble of Section 3.

Scripts used

Vitis HLS script:

```

1 open_project proj_correlation_2D
2 set_top correlation_2D
3 add_files cpu_functions.cpp -cflags "-DFLOAT"
4 open_solution "solution"
5 set_part xc7z020clg484-1
6 create_clock -period 10.0 -name default
7 csynth_design
8 exit

```

Vivado script:

```

1 create_project vivado_correlation_2D ./vivado_correlation_2D -part
   xc7z020clg484-1 -force
2 file mkdir vivado_reports
3 foreach ip_tcl [glob -nocomplain ./proj_correlation_2D/solution/syn/
   verilog/*_ip.tcl] {

```

```

4   source $ip_tcl
5 }
6 read_verilog [glob ./proj_correlation_2D/solution/syn/verilog/*.v]
7 generate_target all [get_ips]
8 synth_design -top correlation_2D -part xc7z020clg484-1
9 create_clock -period 10.0 -name default [get_ports ap_clk]
10 report_utilization -file vivado_reports/utilization_synth.rpt
11 report_timing_summary -file vivado_reports/timing_synth.rpt
12 exit

```

Synthesis Results

A summary of the resource utilization and timing results is shown in Table 5.

Resource	Used	Available	Utilization (%)
LUT	4736	53200	8.90
LUTRAM	117	17400	0.67
FF	5776	106400	5.43
BRAM	0	140	0.00
DSP	26	220	11.82
I/O	135	200	67.50
Frequency (MHz)	100.0, period = 10 ns		
WNS	1.668 ns		

Table 5: Vivado results for `correlation_2D` (Zynq-7020).

3.6 fast_fourier_transform_2D_bench

This benchmark implements a 2-D discrete Fourier transform of a complex-valued input matrix of size $nx \times ny$. The HLS synthesis target is the FFT2D function, which originally delegates the computation to the `fftw_plan_dft_2d` routine from the FFTW3 library.

Compilation Issues and Fixes

- **External library dependency (fftw3):** The original FFT2D function relies on the `fftw3` library, using dynamic constructs such as `fftw_plan`, `fftw_malloc`, and `fftw_execute` that cannot be synthesized by Vitis HLS.
- **Unsupported pointer-to-pointer interface:** Vitis HLS does not support `COMPLEX**` as a top-level interface port, reporting `Pointer to pointer is not supported`.
- **Solution adopted:** Following the same Cooley–Tukey radix-2 DIT strategy used in the Bambu report, a synthesizable reimplementaion was created. The `fftw3` dependency was replaced with a plain C 1-D FFT applied first to every row and then to every column, using a statically allocated column buffer (`COMPLEX col[MAX_N]`) to avoid dynamic memory allocation. Unlike the Bambu version, which required custom Taylor series approximations for `cos` and `sin`, Vitis HLS supports `cosf` and `sinf` natively. The `COMPLEX**` interface was replaced with flat 1-D arrays (`COMPLEX*`) with 2-D indexing expressed as `i*ny + j`, and the top-level function was renamed `FFT2D_flat` to reflect the interface change.

Listing 1: Modified FFT2D (`cpu_functions_vitis.cpp`)

```

1 static void fft1D(COMPLEX* row, int n) {
2     int j = 0;
3     for (int i = 1; i < n; i++) {

```

```

4      int bit = n >> 1;
5      for (; j & bit; bit >>= 1) j ^= bit;
6      j ^= bit;
7      if (i < j) {
8          bench_t tmp_x = row[i].x; row[i].x = row[j].x; row[j].x =
          tmp_x;
9          bench_t tmp_y = row[i].y; row[i].y = row[j].y; row[j].y =
          tmp_y;
10     }
11 }
12 for (int len = 2; len <= n; len <= 1) {
13     bench_t ang = (bench_t)(-2.0 * M_PI / len);
14     bench_t wcos = cosf(ang);
15     bench_t wsine = sinf(ang);
16     for (int i = 0; i < n; i += len) {
17         bench_t cur_cos = 1.0, cur_sine = 0.0;
18         for (int k = 0; k < len / 2; k++) {
19             bench_t ux = row[i+k].x, uy = row[i+k].y;
20             bench_t vx = row[i+k+len/2].x*cur_cos - row[i+k+len/2].y
                *cur_sine;
21             bench_t vy = row[i+k+len/2].x*cur_sine + row[i+k+len/2].y
                *cur_cos;
22             row[i+k].x = ux+vx;          row[i+k].y = uy+vy;
23             row[i+k+len/2].x = ux-vx;    row[i+k+len/2].y = uy-vy;
24             bench_t nc = cur_cos*wcos - cur_sine*wsine;
25             bench_t ns = cur_cos*wsine + cur_sine*wcos;
26             cur_cos = nc; cur_sine = ns;
27         }
28     }
29 }
30 }
31
32 bool FFT2D_flat(COMPLEX* c_flat, int nx, int ny, COMPLEX* out_flat) {
33     for (int i = 0; i < nx * ny; i++)
34         out_flat[i] = c_flat[i];
35     for (int i = 0; i < nx; i++)
36         fft1D(&out_flat[i * ny], ny);
37     COMPLEX col[MAX_N];
38     for (int j = 0; j < ny; j++) {
39         for (int i = 0; i < nx; i++) col[i] = out_flat[i*ny + j];
40         fft1D(col, nx);
41         for (int i = 0; i < nx; i++) out_flat[i*ny + j] = col[i];
42     }
43     return true;
44 }

```

Scripts used

Vitis HLS script:

```

1 open_project proj_fft2d
2 set_top FFT2D_flat
3 add_files cpu_functions_vitis.cpp -cflags "-DFLOAT"
4 open_solution "fsolution"
5 set_part xc7z020clg484-1
6 create_clock -period 10.0 -name default
7 csynth_design
8 exit

```

Vivado script:

```

1 create_project vivado_fft2d ./vivado_fft2d -part xc7z020clg484-1 -force
2 file mkdir vivado_reports
3 foreach ip_tcl [glob -nocomplain ./proj_fft2d/fsolution/syn/verilog/*_ip
  .tcl] {
4     source $ip_tcl
5 }
6 read_verilog [glob ./proj_fft2d/fsolution/syn/verilog/*.v]
7 synth_design -top FFT2D_flat -part xc7z020clg484-1
8 create_clock -period 10.0 -name default [get_ports ap_clk]
9 report_utilization -file vivado_reports/utilization_synth.rpt
10 report_timing_summary -file vivado_reports/timing_synth.rpt
11 exit

```

Synthesis Results

A summary of the resource utilization and timing results is shown in Table 6.

Resource	Used	Available	Utilization (%)
LUT	8509	53200	15.99
LUTRAM	106	17400	0.61
FF	11853	106400	11.14
BRAM	2	140	1.43
DSP	45	220	20.45
I/O	264	200	132.00
Frequency (MHz)	100.0, period = 10 ns		
WNS	1.668 ns		

Table 6: Vivado results for FFT2D_flat (Zynq-7020).

3.7 fast_fourier_transform_bench

This benchmark implements a 1-D discrete Fourier transform of a real-valued input array using the Danielson–Lanczos variant of the Cooley–Tukey radix-2 algorithm, directly in C++ without relying on any external FFT library.

Compilation Issues and Fixes

No issues were reported.

Scripts used

Vitis HLS script:

```

1 open_project proj_fft
2 set_top fft_function
3 add_files cpu_functions.cpp -cflags "-DFLOAT"
4 open_solution "fsolution"
5 set_part xc7z020clg484-1
6 create_clock -period 10.0 -name default
7 csynth_design
8 exit

```

Vivado script:

```

1 create_project vivado_fft ./vivado_fft -part xc7z020clg484-1 -force
2 file mkdir vivado_reports
3 foreach ip_tcl [glob -nocomplain ./proj_fft/fsolution/syn/verilog/*_ip.
  tcl] {
4     source $ip_tcl
5 }
6 read_verilog [glob ./proj_fft/fsolution/syn/verilog/*.v]
7 synth_design -top fft_function -part xc7z020clg484-1
8 create_clock -period 10.0 -name default [get_ports ap_clk]
9 report_utilization -file vivado_reports/utilization_synth.rpt
10 report_timing_summary -file vivado_reports/timing_synth.rpt
11 exit

```

Synthesis Results

A summary of the resource utilization and timing results is shown in Table 7.

Resource	Used	Available	Utilization (%)
LUT	10272	53200	19.31
LUTRAM	124	17400	0.71
FF	12283	106400	11.54
BRAM	0	140	0.00
DSP	122	220	55.45
I/O	135	200	67.50
Frequency (MHz)	100.0, period = 10 ns		
WNS	1.668 ns		

Table 7: Vivado results for `fft_function` from `fast_fourier_transform_bench` (Zynq-7020).

3.8 fast_fourier_transform_window_bench

This benchmark computes a windowed FFT over the input signal. The HLS target is the `fft_function`, which copies each input window to the output buffer and then applies the FFT.

Compilation Issues and Fixes

No issues were reported.

Scripts used

The same scripts as in `fast_fourier_transform_bench` were used, with the project and output directories renamed accordingly (`proj_fft_window`, `vivado_fft_window`).

Synthesis Results

A summary of the resource utilization and timing results is shown in Table 8.

Resource	Used	Available	Utilization (%)
LUT	10407	53200	19.56
LUTRAM	124	17400	0.71
FF	12572	106400	11.82
BRAM	0	140	0.00
DSP	122	220	55.45
I/O	230	200	115.00
Frequency (MHz)	100.0, period = 10 ns		
WNS	1.668 ns		

Table 8: Vivado results for `fft_function` from `fast_fourier_transform_window_bench` (Zynq-7020).

3.9 finite_impulse_response_filter

This benchmark computes a Finite Impulse Response (FIR) filter over an input signal. The HLS target is the `vector_convolution` function, which slides the filter kernel over the input signal and accumulates the weighted sum of samples at each output position.

Compilation Issues and Fixes

No issues were reported.

Scripts used

Vitis HLS script:

```

1 open_project proj_fir
2 set_top vector_convolution
3 add_files cpu_functions.cpp -cflags "-DFLOAT"
4 open_solution "fsolution"
5 set_part xc7z020clg484-1
6 create_clock -period 10.0 -name default
7 csynth_design
8 exit

```

For this benchmark, other flags such as `-DINT` and `-DDOUBLE` can also be used.

Vivado script:

```

1 create_project vivado_fir ./vivado_fir -part xc7z020clg484-1 -force
2 file mkdir vivado_reports
3 foreach ip_tcl [glob -nocomplain ./proj_fir/fsolution/syn/verilog/*_ip.
   tcl] {
4     source $ip_tcl
5 }
6 read_verilog [glob ./proj_fir/fsolution/syn/verilog/*.v]
7 synth_design -top vector_convolution -part xc7z020clg484-1
8 create_clock -period 10.0 -name default [get_ports ap_clk]
9 report_utilization -file vivado_reports/utilization_synth.rpt
10 report_timing_summary -file vivado_reports/timing_synth.rpt
11 exit

```

Synthesis Results

A summary of the resource utilization and timing results is shown in Table 9.

Resource	Available	-DINT		-DFLOAT		-DDOUBLE	
		Used	Util. (%)	Used	Util. (%)	Used	Util. (%)
LUT	53200	374	0.70	568	1.07	1061	1.99
LUTRAM	17400	1	0.01	0	0.00	30	0.17
FF	106400	515	0.48	822	0.77	1516	1.42
BRAM	140	0	0.00	0	0.00	0	0.00
DSP	220	7	3.18	9	4.09	18	8.18
I/O	200	199	99.50	199	99.50	327	163.50
Frequency (MHz)	—	100.0 (10 ns)		100.0 (10 ns)		100.0 (10 ns)	
WNS (ns)	—	3.858		1.668		2.452	

Table 9: Vivado results for `vector_convolution` (Zynq-7020).

3.10 matrix_multiplication_bench

This benchmark computes the multiplication of two square matrices of size $N \times N$. The HLS target is the `matrix_multiplication` function, which implements the standard triple-nested loop algorithm.

Compilation Issues and Fixes

No issues were reported.

Scripts used

Vitis HLS script:

```
1 open_project proj_matmul
2 set_top matrix_multiplication
3 add_files cpu_functions.cpp -cflags "-DFLOAT"
4 open_solution "fsolution"
5 set_part xc7z020clg484-1
6 create_clock -period 10.0 -name default
7 csynth_design
8 exit
```

For this benchmark, other flags such as `-DINT` and `-DDOUBLE` can also be used.

Vivado script:

```
1 create_project vivado_matmul ./vivado_matmul -part xc7z020clg484-1 -
  force
2 file mkdir vivado_reports
3 foreach ip_tcl [glob -nocomplain ./proj_matmul/fsolution/syn/verilog/*
  _ip.tcl] {
4   source $ip_tcl
5 }
6 read_verilog [glob ./proj_matmul/fsolution/syn/verilog/*.v]
7 synth_design -top matrix_multiplication -part xc7z020clg484-1
8 create_clock -period 10.0 -name default [get_ports ap_clk]
9 report_utilization -file vivado_reports/utilization_synth.rpt
10 report_timing_summary -file vivado_reports/timing_synth.rpt
```

Synthesis Results

A summary of the resource utilization and timing results is shown in Table 10.

Resource	Available	-DINT		-DFLOAT		-DDOUBLE	
		Used	Util. (%)	Used	Util. (%)	Used	Util. (%)
LUT	53200	101	0.19	466	0.88	993	1.87
LUTRAM	17400	0	0.00	0	0.00	30	0.17
FF	106400	233	0.22	692	0.65	1389	1.31
BRAM	140	0	0.00	0	0.00	0	0.00
DSP	220	6	2.73	5	2.27	14	6.36
I/O	200	231	115.50	231	115.50	359	179.50
Frequency (MHz)	–	100.0 (10 ns)		100.0 (10 ns)		100.0 (10 ns)	
WNS (ns)	–	4.159		1.668		2.452	

Table 10: Vivado results for `matrix_multiplication` from `matrix_multiplication_bench` (Zynq-7020).

3.11 `matrix_multiplication_bench_fp16`

This benchmark computes the multiplication of two square matrices, analogously to the previous `matrix_multiplication_bench`. The key difference is that this variant targets half-precision floating-point (FP16) arithmetic, intended for GPU execution using CUDA `half` types. The HLS target is again the `matrix_multiplication` function.

Compilation Issues and Fixes

No issues were reported.

Scripts used

The same scripts as in Section 3.10 were used.

Synthesis Results

The synthesized RTL is identical to that of `matrix_multiplication_bench`, since the FP16 distinction exists only at the GPU execution level. Vitis HLS resolves the CPU reference implementation to the same `float`-precision function and produces an identical netlist. Results are therefore identical to those in Table 10.

3.12 `matrix_multiplication_tensor_bench`

This benchmark targets GPU tensor cores for mixed-precision matrix operations. The HLS target is again the `matrix_multiplication` function.

Compilation Issues and Fixes

No issues were reported.

Scripts used

The same scripts as in Section 3.10 were used.

Synthesis Results

As with `matrix_multiplication_bench_fp16`, the tensor core distinction exists exclusively at the GPU level and the CPU reference implementation resolves to the same function. The synthesized netlist is therefore identical to that of `matrix_multiplication_bench`, in Table 10.

3.13 max_pooling_bench

This benchmark computes a max pooling operation over a 2D input matrix, a fundamental building block in convolutional neural networks. The HLS target is the `max_pooling` function, which slides a window of size `stride` $\times$ `stride` over the input matrix and writes the maximum value of each window to the output.

Compilation Issues and Fixes

No issues were reported.

Scripts used

Vitis HLS script:

```
1 open_project proj_maxpool
2 set_top max_pooling
3 add_files cpu_functions.cpp -cflags "-DFLOAT"
4 open_solution "fsolution"
5 set_part xc7z020clg484-1
6 create_clock -period 10.0 -name default
7 csynth_design
8 exit
```

For this benchmark, other flags such as `-DINT` and `-DDOUBLE` can also be used.

Vivado script:

```
1 create_project vivado_maxpool ./vivado_maxpool -part xc7z020clg484-1 -
  force
2 file mkdir vivado_reports
3 foreach ip_tcl [glob -nocomplain ./proj_maxpool/fsolution/syn/verilog/*
  _ip.tcl] {
4   source $ip_tcl
5 }
6 read_verilog [glob ./proj_maxpool/fsolution/syn/verilog/*.v]
7 synth_design -top max_pooling -part xc7z020clg484-1
8 create_clock -period 10.0 -name default [get_ports ap_clk]
9 report_utilization -file vivado_reports/utilization_synth.rpt
10 report_timing_summary -file vivado_reports/timing_synth.rpt
11 exit
```

Synthesis Results

A summary of the resource utilization and timing results is shown in Table 11.

Resource	Available	-DINT		-DFLOAT		-DDOUBLE	
		Used	Util. (%)	Used	Util. (%)	Used	Util. (%)
LUT	53200	197	0.37	330	0.62	328	0.62
LUTRAM	17400	0	0.00	0	0.00	0	0.00
FF	106400	396	0.37	531	0.50	661	0.62
BRAM	140	0	0.00	0	0.00	0	0.00
DSP	220	4	1.82	4	1.82	4	1.82
I/O	200	135	67.5	135	67.5	199	99.5
Frequency (MHz)	—	100.0 (10 ns)		100.0 (10 ns)		100.0 (10 ns)	
WNS (ns)	—	4.858		4.519		4.710	

Table 11: Vivado results for `max_pooling` (Zynq-7020).

3.14 memory_bandwidth_bench

This benchmark measures memory bandwidth by performing a device-to-device memory copy via `cudaMemcpy`, with no synthesizable computational kernel in the original source. Following feedback from the project supervisor, a plain-C `copy_array` function was implemented from scratch as the HLS target.

Compilation Issues and Fixes

- **No synthesizable HLS target in the original source:** The original benchmark relies exclusively on CUDA memory transfer operations (`cudaMemcpy`), which cannot be synthesized by Vitis HLS.
- **Solution adopted:** A new `cpu_functions/` subdirectory was created containing a synthesizable C implementation of the copy operation, used as the Vitis HLS top function.

Listing 2: Created header `cpu_functions.h`

```

1 #ifndef CPU_FUNCTIONS_H
2 #define CPU_FUNCTIONS_H
3
4 #ifdef INT
5     typedef int bench_t;
6 #elif FLOAT
7     typedef float bench_t;
8 #elif DOUBLE
9     typedef double bench_t;
10 #endif
11
12 void copy_array(bench_t *A, bench_t *B, int size);
13
14 #endif

```

Listing 3: `copy_array` function (`cpu_functions.cpp`)

```

1 void copy_array(bench_t *A, bench_t *B, int size) {
2     for (int i = 0; i < size; i++) {
3         B[i] = A[i];
4     }
5 }

```

Scripts used

Vitis HLS script:

```
1 open_project proj_membw
2 set_top copy_array
3 add_files cpu_functions.cpp -cflags "-DFLOAT"
4 open_solution "fsolution"
5 set_part xc7z020clg484-1
6 create_clock -period 10.0 -name default
7 csynth_design
8 exit
```

Vivado script:

```
1 create_project vivado_membw ./vivado_membw -part xc7z020clg484-1 -force
2 file mkdir vivado_reports
3 foreach ip_tcl [glob -nocomplain ./proj_membw/fsolution/syn/verilog/*_ip
4   .tcl] {
5     source $ip_tcl
6 }
7 read_verilog [glob ./proj_membw/fsolution/syn/verilog/*.v]
8 synth_design -top copy_array -part xc7z020clg484-1
9 report_utilization -file vivado_reports/utilization_synth.rpt
10 report_timing_summary -file vivado_reports/timing_synth.rpt
11 exit
```

Synthesis Results

A summary of the resource utilization and timing results is shown in Table 12.

Resource	Available	-DINT		-DFLOAT		-DDOUBLE	
		Used	Util. (%)	Used	Util. (%)	Used	Util. (%)
LUT	53200	17	0.03	17	0.03	17	0.03
LUTRAM	17400	0	0.00	0	0.00	0	0.00
FF	106400	0	0.00	0	0.00	0	0.00
BRAM	140	0	0.00	0	0.00	0	0.00
DSP	220	0	0.00	0	0.00	0	0.00
I/O	200	101	50.50	101	50.50	165	82.50
Frequency (MHz)	—	—		—		—	
WNS (ns)	—	—		—		—	

Table 12: Vivado results for `copy_array` (Zynq-7020).

The synthesized design does not expose an `ap_clk` port: Vitis HLS optimizes the simple copy loop into a purely combinational datapath, removing all control logic and registers. No clock constraint can therefore be applied and timing metrics are not reported.

3.15 relu_bench

This benchmark computes the Rectified Linear Unit (ReLU) activation function over a 2D input matrix. The HLS target is the `relu` function, which sets each element to the input value if positive, and to zero otherwise.

Compilation Issues and Fixes

No issues were reported.

Scripts used

Vitis HLS script:

```
1 open_project proj_relu
2 set_top relu
3 add_files cpu_functions.cpp -cflags "-DFLOAT"
4 open_solution "fsolution"
5 set_part xc7z020clg484-1
6 create_clock -period 10.0 -name default
7 csynth_design
8 exit
```

For this benchmark, other flags such as `-DINT` and `-DDOUBLE` can also be used.

Vivado script:

```
1 create_project vivado_relu ./vivado_relu -part xc7z020clg484-1 -force
2 file mkdir vivado_reports
3 foreach ip_tcl [glob -nocomplain ./proj_relu/fsolution/syn/verilog/*_ip.
   tcl] {
4     source $ip_tcl
5 }
6 read_verilog [glob ./proj_relu/fsolution/syn/verilog/*.v]
7 synth_design -top relu -part xc7z020clg484-1
8 create_clock -period 10.0 -name default [get_ports ap_clk]
9 report_utilization -file vivado_reports/utilization_synth.rpt
10 report_timing_summary -file vivado_reports/timing_synth.rpt
11 exit
```

Synthesis Results

A summary of the resource utilization and timing results is shown in Table 13.

Resource	Available	-DINT		-DFLOAT		-DDOUBLE	
		Used	Util. (%)	Used	Util. (%)	Used	Util. (%)
LUT	53200	125	0.23	125	0.23	206	0.39
LUTRAM	17400	0	0.00	0	0.00	0	0.00
FF	106400	235	0.22	235	0.22	364	0.34
BRAM	140	0	0.00	0	0.00	0	0.00
DSP	220	4	1.82	4	1.82	4	1.82
I/O	200	103	51.05	103	51.05	167	83.50
Frequency (MHz)	—	100.0 (10 ns)		100.0 (10 ns)		100.0 (10 ns)	
WNS (ns)	—	5.398		5.408		5.394	

Table 13: Vivado results for `relu` (Zynq-7020).

3.16 softmax_bench

This benchmark computes the Softmax activation function over a 2D input matrix. The HLS target is the `softmax` function, which computes the exponential of each element, accumulates the total sum, and normalizes each element by dividing by that sum.

Compilation Issues and Fixes

No issues beyond the standard mathematical IP generation described in the preamble of Section 3.

Scripts used

Vitis HLS script:

```
1 open_project proj_softmax
2 set_top softmax
3 add_files cpu_functions.cpp -cflags "-DFLOAT"
4 open_solution "solution"
5 set_part xc7z020clg484-1
6 create_clock -period 10.0 -name default
7 csynth_design
8 exit
```

Vivado script:

```
1 create_project vivado_softmax ./vivado_softmax -part xc7z020clg484-1 -
  force
2 file mkdir vivado_reports
3 foreach ip_tcl [glob -nocomplain ./proj_softmax/solution/syn/verilog/*
  _ip.tcl] {
4   source $ip_tcl
5 }
6 read_verilog [glob ./proj_softmax/solution/syn/verilog/*.v]
7 generate_target all [get_ips]
8 synth_design -top softmax -part xc7z020clg484-1 -mode out_of_context
9 create_clock -period 10.0 -name default [get_ports ap_clk]
10 report_utilization -file vivado_reports/utilization_synth.rpt
11 report_timing_summary -file vivado_reports/timing_synth.rpt
12 exit
```

Synthesis Results

A summary of the resource utilization and timing results is shown in Table 14.

Resource	Used	Available	Utilization (%)
LUT	3273	53200	6.15
LUTRAM	230	17400	1.32
FF	2720	106400	2.56
BRAM	0	140	0.00
DSP	32	220	14.55
I/O	135	200	67.5
Frequency (MHz)	100.0, period = 10 ns		
WNS	1.668 ns		

Table 14: Vivado results for **softmax** (Zynq-7020).

3.17 wavelet_transform

This benchmark computes the CCSDS Wavelet Transform over a 1D input signal. The HLS target is the `ccsds_wavelet_transform` function, which applies a lowpass and a highpass filter to the input signal.

Compilation Issues and Fixes

- **Invalid memory access on scalar ports:** Vitis HLS inferred the input and output arrays as scalar (register) interfaces, while the design performs indexed accesses (e.g., `A[2*i+1]`), causing address computation errors during synthesis.
- **Solution adopted:** Explicit `m_axi` memory-mapped AXI interfaces were defined for the input and output arrays, with control signals mapped to an `s_axilite` interface. The design was then synthesized in OOC mode to avoid physical I/O mapping issues arising from the AXI ports.

Scripts used

Vitis HLS script:

```
1 open_project proj_wavelet
2 set_top ccsds_wavelet_transform
3 add_files cpu_functions.cpp -cflags "-DFLOAT"
4 open_solution "solution"
5 set_part xc7z020clg484-1
6 create_clock -period 10.0 -name default
7 csynth_design
8 exit
```

For this benchmark, other flags such as `-DINT` and `-DDOUBLE` can also be used.

Vivado script:

```
1 create_project vivado_wavelet ./vivado_wavelet -part xc7z020clg484-1 -
  force
2 file mkdir vivado_reports
3 foreach ip_tcl [glob -nocomplain ./proj_wavelet/solution/syn/verilog/*
  _ip.tcl] {
4     source $ip_tcl
5 }
6 read_verilog [glob ./proj_wavelet/solution/syn/verilog/*.v]
7 generate_target all [get_ips]
8 synth_design -top ccsds_wavelet_transform -part xc7z020clg484-1 -mode
  out_of_context
9 create_clock -period 10.0 -name default [get_ports ap_clk]
10 report_utilization -file vivado_reports/utilization_synth.rpt
11 report_timing_summary -file vivado_reports/timing_synth.rpt
12 exit
```

Synthesis Results

A summary of the resource utilization and timing results is shown in Table 15.

Resource	Available	-DINT		-DFLOAT		-DDOUBLE	
		Used	Util. (%)	Used	Util. (%)	Used	Util. (%)
LUT	53200	7179	13.49	4654	8.75	6426	12.08
LUTRAM	17400	422	2.43	420	2.41	741	4.26
FF	106400	9295	8.74	5694	5.35	8510	8.00
BRAM	140	1	0.71	0.5	0.36	1	0.71
DSP	220	28	12.73	10	4.55	28	12.73
I/O	200	640	320.00	605	302.50	709	354.50
Frequency (MHz)	—	100.0 (10 ns)		100.0 (10 ns)		100.0 (10 ns)	
WNS (ns)	—	2.452		1.668		2.452	

Table 15: Vivado results for `ccsds_wavelet_transform` (Zynq-7020).

4 Results Summary & Discussion

Table 16 shows the synthesis results obtained with Vitis HLS and Vivado for each benchmark using the -DFLOAT configuration. For benchmarks synthesized in out-of-context mode, the reported I/O value corresponds to logical top-level ports and is therefore not directly comparable to physical FPGA pin utilization.

Benchmark	LUT	LUTRAM	FF	BRAM	DSP	I/O	Freq. (MHz)	WNS (ns)
Available	53200	17400	106400	140	220	200	–	–
LRN_bench	1662	0	1750	10.5	103	103	103.5	5.776
cifar_10	21424	1879	30349	11	136	242	100.0	-1.092
cifar_10_multiple	26662	1746	33767	10.5	160	844	100.0	1.393
convolution_2D_bench	1669	0	2956	0	41	167	100.0	1.590
correlation_2D	4736	117	5776	0	26	135	100.0	1.668
fft_2D_bench	8509	106	11853	2	45	264	100.0	1.668
fft_bench	10272	124	12283	0	122	135	100.0	1.668
fft_window_bench	10407	124	12572	0	122	230	100.0	1.668
fir_filter	568	0	822	0	9	199	100.0	1.668
matrix_mult_bench	466	0	692	0	5	231	100.0	1.668
matrix_mult_fp16	466	0	692	0	5	231	100.0	1.668
matrix_mult_tensor	466	0	692	0	5	231	100.0	1.668
max_pooling_bench	330	0	531	0	4	135	100.0	4.519
memory_bandwidth_bench	17	0	0	0	0	101	–	–
relu_bench	125	0	235	0	4	103	100.0	5.408
softmax_bench	3273	230	2720	0	32	135	100.0	1.668
wavelet_transform	4654	420	5694	0.5	10	605	100.0	1.668

Table 16: Results summary for all GPU4S_Bench benchmarks on the Zynq-7020 FPGA with the -DFLOAT version using Vitis HLS.

Benchmarks excluded from datatypes config. Not all GPU4S Bench kernels were evaluated under the three datatype configurations (-DINT, -DFLOAT, and -DDOUBLE). In several cases, a datatype variant was intentionally excluded. Table 19 in Appendix B summarizes the excluded configurations and their justification.

4.1 Comparison with Bambu

Tables 17–31 compare the -DFLOAT results obtained with Vitis HLS against the Bambu results reported in the previous report.

Tool	LUT	LUTRAM	FF	BRAM	DSP	WNS (ns)
Vitis HLS	1662 (3.12%)	0 (0.00%)	1750 (1.64%)	10.5 (7.50%)	103 (46.82%)	5.776
Bambu	1662 (3.12%)	0 (0.00%)	1750 (1.64%)	0 (0.00%)	5 (2.27%)	0.340

Table 17: Comparison for LRN_bench.

Tool	LUT	LUTRAM	FF	BRAM	DSP	WNS (ns)
Vitis HLS	21424 (40.27%)	1879 (10.80%)	30349 (28.52%)	11 (7.86%)	136 (61.82%)	-1.092
Bambu	9698 (18.23%)	0 (0.00%)	7975 (7.50%)	0 (0.00%)	126 (57.27%)	-0.215

Table 18: Comparison for cifar_10.

Tool	LUT	LUTRAM	FF	BRAM	DSP	WNS (ns)
Vitis HLS	26662 (50.12%)	1746 (10.03%)	33767 (31.74%)	10.5 (7.50%)	160 (72.73%)	1.393
Bambu	9860 (18.53%)	0 (0.00%)	8101 (7.61%)	0 (0.00%)	132 (60.00%)	-0.576

Table 19: Comparison for `cifar_10_multiple`.

Tool	LUT	LUTRAM	FF	BRAM	DSP	WNS (ns)
Vitis HLS	1669 (3.14%)	0 (0.00%)	2956 (2.78%)	0 (0.00%)	41 (18.64%)	1.590
Bambu	1669 (3.14%)	0 (0.00%)	1277 (1.20%)	0 (0.00%)	11 (5.00%)	0.091

Table 20: Comparison for `convolution_2D_bench`.

Tool	LUT	LUTRAM	FF	BRAM	DSP	WNS (ns)
Vitis HLS	4736 (8.90%)	117 (0.67%)	5776 (5.43%)	0 (0.00%)	26 (11.82%)	1.668
Bambu	3791 (7.13%)	0 (0.00%)	2902 (2.73%)	0 (0.00%)	14 (6.36%)	-0.219

Table 21: Comparison for `correlation_2D`.

Tool	LUT	LUTRAM	FF	BRAM	DSP	WNS (ns)
Vitis HLS	8509 (15.99%)	106 (0.61%)	11853 (11.14%)	2 (1.43%)	45 (20.45%)	1.668
Bambu	6894 (12.96%)	0 (0.00%)	5044 (4.74%)	1 (0.71%)	8 (3.64%)	-1.064

Table 22: Comparison for `fft_2D_bench`.

Tool	LUT	LUTRAM	FF	BRAM	DSP	WNS (ns)
Vitis HLS	10272 (19.31%)	124 (0.71%)	12283 (11.54%)	0 (0.00%)	122 (55.45%)	1.668
Bambu	7719 (14.51%)	0 (0.00%)	5725 (5.38%)	0 (0.00%)	18 (8.18%)	-0.689

Table 23: Comparison for `fft_bench`.

Tool	LUT	LUTRAM	FF	BRAM	DSP	WNS (ns)
Vitis HLS	10407 (19.56%)	124 (0.71%)	12572 (11.82%)	0 (0.00%)	122 (55.45%)	1.668
Bambu	8230 (15.47%)	0 (0.00%)	6032 (5.67%)	0 (0.00%)	24 (10.91%)	-0.325

Table 24: Comparison for `fft_window_bench`.

Tool	LUT	LUTRAM	FF	BRAM	DSP	WNS (ns)
Vitis HLS	568 (1.07%)	0 (0.00%)	822 (0.77%)	0 (0.00%)	9 (4.09%)	1.668
Bambu	933 (1.75%)	0 (0.00%)	727 (0.68%)	0 (0.00%)	2 (0.91%)	0.184

Table 25: Comparison for `fir_filter`.

Tool	LUT	LUTRAM	FF	BRAM	DSP	WNS (ns)
Vitis HLS	466 (0.88%)	0 (0.00%)	692 (0.65%)	0 (0.00%)	5 (2.27%)	1.668
Bambu	913 (1.72%)	0 (0.00%)	574 (0.54%)	0 (0.00%)	8 (3.64%)	0.214

Table 26: Comparison for `matrix_multiplication`.

Tool	LUT	LUTRAM	FF	BRAM	DSP	WNS (ns)
Vitis HLS	330 (0.62%)	0 (0.00%)	531 (0.50%)	0 (0.00%)	4 (1.82%)	4.519
Bambu	1329 (2.50%)	0 (0.00%)	883 (0.83%)	0 (0.00%)	33 (15.00%)	-0.225

Table 27: Comparison for `max_pooling`.

Tool	LUT	LUTRAM	FF	BRAM	DSP	WNS (ns)
Vitis HLS	17 (0.03%)	0 (0.00%)	0 (0.00%)	0 (0.00%)	0 (0.00%)	—
Bambu	162 (0.30%)	0 (0.00%)	104 (0.10%)	0 (0.00%)	0 (0.00%)	5.311

Table 28: Comparison for `memory_bandwidth_bench`.

Tool	LUT	LUTRAM	FF	BRAM	DSP	WNS (ns)
Vitis HLS	125 (0.23%)	0 (0.00%)	235 (0.22%)	0 (0.00%)	4 (1.82%)	5.408
Bambu	298 (0.56%)	0 (0.00%)	153 (0.14%)	0 (0.00%)	3 (1.36%)	4.462

Table 29: Comparison for `relu_bench`.

Tool	LUT	LUTRAM	FF	BRAM	DSP	WNS (ns)
Vitis HLS	3273 (6.15%)	230 (1.32%)	2720 (2.56%)	0 (0.00%)	32 (14.55%)	1.668
Bambu	1476 (2.77%)	0 (0.00%)	1418 (1.33%)	0 (0.00%)	8 (3.64%)	0.028

Table 30: Comparison for `softmax`.

Tool	LUT	LUTRAM	FF	BRAM	DSP	WNS (ns)
Vitis HLS	4654 (8.75%)	420 (2.41%)	5694 (5.35%)	0.5 (0.36%)	10 (4.55%)	1.668
Bambu	1328 (2.50%)	0 (0.00%)	1162 (1.09%)	0 (0.00%)	2 (0.91%)	0.146

Table 31: Comparison for `wavelet_transform`.

4.2 Description of the results

All evaluated GPU4S_Bench kernels were successfully synthesized with Vitis HLS and then processed with Vivado. Compared with the previous Bambu flow, the Vitis flow required fewer algorithmic approximations for mathematical functions, since operations such as `exp()`, `sqrt()`, `cosf()`, and `sinf()` can be mapped to generated floating-point IP cores. This is especially relevant for kernels such as `softmax`, `correlation_2D`, and the FFT benchmarks, where Bambu required manual replacements.

Effect of the datatype configuration

The three datatype configurations produce noticeably different synthesis results depending on the arithmetic requirements of each kernel. The benchmarks evaluated under all configurations are `convolution_2D_bench`, `fir_filter`, `matrix_multiplication_bench`, `max_pooling_bench`, `relu_bench`, `wavelet_transform`, and `memory_bandwidth_bench`. The remaining kernels were restricted to `-DFLOAT` due to algorithmic constraints documented in Appendix B.

Integer (`-DINT`) yields the most compact designs, as integer arithmetic maps directly onto carry-chain logic without floating-point IP overhead. For example, `matrix_multiplication` drops from 466 LUTs (`-DFLOAT`) to just 101 LUTs (`-DINT`).

Single-precision (`-DFLOAT`) increases FF and DSP usage significantly, since Vitis HLS instantiates multi-stage pipelined IP cores for each floating-point operation. The effect is most visible in arithmetic-heavy kernels such as `convolution_2D_bench`, which grows from 2588 to 2956 FFs.

Double-precision (`-DDOUBLE`) pushes resource consumption further due to wider 64-bit data paths, with LUT and DSP counts roughly doubling for compute-intensive kernels (`fir_filter`: 568 $\rightarrow$ 1061 LUTs, 9 $\rightarrow$ 18 DSPs). Timing slack does not degrade monotonically: some kernels show better WNS under `-DDOUBLE` than `-DFLOAT`, likely due to differences in how double-precision IP cores structure their internal pipelines.

Finally, `relu_bench` and `max_pooling_bench` are largely insensitive to the datatype configuration in terms of LUT and DSP usage, since their operations (element-wise comparison and selection) do not require complex arithmetic pipelines regardless of precision.

Comparison across benchmarks (`-DFLOAT`)

Examining the `-DFLOAT` results (Table 16), a clear stratification emerges based on algorithmic complexity.

Minimal designs (`memory_bandwidth_bench`, `relu_bench`, `max_pooling_bench`) perform only element-wise operations and consume fewer than 350 LUTs and 4 DSPs. `memory_bandwidth_bench` is the extreme case, reduced to purely combinational logic with no registers or clock port.

Small-to-medium designs (`matrix_multiplication`, `fir_filter`, `convolution_2D_bench`) range from 466 to 1669 LUTs. `LRN_bench` fits here in LUT count (1662) but stands out with 103 DSPs (46.82%), driven by its normalization and exponentiation operations.

Large designs are the FFT family and CIFAR-10 variants, consuming 8500–26600 LUTs and 45–160 DSPs. The 1-D FFT benchmarks use notably more DSPs than `fft_2D_bench` due to the higher twiddle-factor multiplication count in the Danielson–Lanczos algorithm. The CIFAR-10 variants are the most demanding overall; `cifar_10` is the only design that fails timing (WNS = -1.092 ns).

BRAM usage is limited to five kernels (`LRN_bench`, `cifar_10`, `cifar_10_multiple`, `fft_2D_bench`, `wavelet_transform`), all of which require intermediate storage beyond what distributed registers can provide.

Comparison with Bambu

The comparison with Bambu reveals two main patterns. First, Vitis adopts a significantly more aggressive hardware mapping strategy, leading to higher resource utilization, especially in DSPs and FFs. This is primarily because Vitis instantiates dedicated floating-point IP cores, whereas Bambu relies on simplified or manually approximated arithmetic implementations.

Second, Vitis generally achieves better timing closure. Bambu reports timing violations for several benchmarks, including `correlation_2D`, `fft_2D_bench`, `fft_bench`, `fft_window_bench`, and `max_pooling_bench`. In contrast, the corresponding Vitis designs meet timing with positive WNS values. The main exception is `cifar_10`, where Vitis still fails timing while also consuming more resources.

Overall, Vitis prioritizes performance and numerical robustness over area efficiency, resulting in a more aggressive use of FPGA resources. Bambu, on the other hand, tends to generate more compact designs, but its limited support for complex mathematical operations often requires manual source modifications, which can negatively impact both accuracy and timing. Therefore, for simple arithmetic kernels, both tools are viable; for math-intensive kernels, Vitis provides better automation and timing reliability at the cost of increased resource usage.

5 Implementation Effort and HLS Constraints

Throughout the development of this project, several challenges emerged regarding the suitability of Bambu for complex algorithmic acceleration. This section summarizes the human effort required and discusses whether the conversion to HLS is justifiable for certain kernels.

5.1 Effort Analysis

Table 32 summarizes the modifications required for the modified benchmarks and the estimated effort. The effort is categorized from Low (basic syntax changes) to Critical (complete algorithmic rewrite).

Benchmark	Modifications Required	Effort	Complexity
<code>cifar_10</code>	OOB synthesis mode for I/O overutilization	Low	★
<code>cifar_10_multiple</code>	OOB synthesis mode (same as <code>cifar_10</code>)	Low	★
<code>fft_2D_bench</code>	FFTW3 replaced with Cooley–Tukey; COMPLEX** replaced with flat 1-D arrays	Critical	★★★★★
<code>softmax_bench</code>	OOB synthesis mode	Low	★
<code>wavelet_transform</code>	Explicit AXI interface directives; OOB synthesis mode	Low	★

Table 32: Summary of modification effort and complexity for adapted benchmarks.

The effort estimates in Table 32 are subjective, as they reflect the perspective of the developer carrying out this work.

5.2 Critical Assessment

A key finding of this work is that Vitis HLS presents a significantly lower entry barrier than Bambu for kernels involving standard mathematical functions. Since Vitis natively supports `exp()`, `sqrt()`, `cosf()`, and `sinf()` through floating-point IP cores, the precision loss and manual approximation effort that characterized the Bambu flow are largely eliminated.

- **Precision:** Unlike Bambu, no Taylor series or Newton–Raphson approximations were required. Vitis maps mathematical functions directly to IEEE 754-compliant IP cores, preserving numerical accuracy without any developer intervention.
- **Development Cost vs. Benefit:** The majority of benchmarks required no source-level modifications at all, which genuinely fulfills the core promise of HLS: rapid transition from software to hardware. The few cases that did require effort (`fft_2D_bench`) were driven by unsupported interfaces or missing kernel implementations, not by tool limitations in arithmetic support.

Recommendation. For most of the GPU4S_Bench kernels, the effort invested with Vitis HLS was clearly justified by the results: designs met timing, required minimal adaptation, and produced numerically correct outputs. The one exception is `cifar_10`, which still fails timing even with Vitis. Compared to Bambu, Vitis represents a more practical choice when mathematical correctness and low adaptation effort are priorities.

6 Conclusions

This report has presented the application of AMD Vitis HLS and Vivado to the full GPU4S_Bench suite, targeting the Zynq-7020 FPGA. All seventeen kernels were successfully synthesized.

Overall, Vitis HLS requires significantly less adaptation effort than Bambu, as native support for transcendental functions eliminates the need for manual approximations. Also, Vitis designs consume more hardware resources than their Bambu counterparts, particularly in DSPs and FFs, due to the pipelined floating-point IP cores instantiated for each arithmetic operation. And finally, timing closure was achieved in all benchmarks except `cifar_10`, which exceeds the capacity of the Zynq-7020 at 100 MHz regardless of the tool used.

7 Future Work

The next step is to compare the results obtained in this report with those from the Bambu-based and the MATLAB-based HLS workflow. All tools will be evaluated in terms of resource utilization, operating frequency, and timing closure across the same benchmarks.

A FPGA Synthesis Metrics

Metric	Description
LUT	Basic configurable logic element. Implements small combinational boolean functions. Reported as <i>LUT as Logic</i> (excluding memory-configured LUTs).
LUTRAM	LUTs configured as distributed RAM instead of logic. Used for lightweight on-chip storage.
FF	Flip-Flop. Sequential element holding one bit of state per clock cycle. Used for registers and pipeline stages.
BRAM	Block RAM. Dedicated hard memory primitives for larger on-chip storage needs.
DSP	Digital Signal Processing slice. Hard arithmetic block optimized for multiply-accumulate operations.
I/O	Input/Output pins (IOBs). Physical interface resources used to connect the design to external signals. High utilization may indicate bandwidth or interface constraints.
Frequency	Operating clock frequency after implementation. Target period is 10 ns (100 MHz) as set by Bambu’s TCL script.
WNS	Worst Negative Slack. Timing margin of the critical path. Positive = timing met; negative = timing violation.

Table 33: Description of the FPGA resource and timing metrics reported by Vivado.

B Excluded Datatype Configurations

Table 34 lists the benchmarks for which one or more datatype configurations (`-DINT` or `-DDOUBLE`) were intentionally not evaluated, together with the reason for their exclusion.

Benchmark	Excluded	Reason
<code>LRN_bench</code>	<code>-DINT</code> <code>-DDOUBLE</code>	The algorithm relies on real-valued computations (normalization, exponentiation). Using <code>-DINT</code> would cause truncation and loss of precision. The Newton–Raphson approximation uses single-precision literals (<code>float</code>), making <code>-DDOUBLE</code> functionally equivalent to <code>-DFLOAT</code> and therefore not representative.
<code>cifar_10</code>	<code>-DINT</code> <code>-DDOUBLE</code>	The CIFAR-10 pipeline internally calls <code>lrn</code> and <code>softmax</code> , both of which depend on real-valued arithmetic and single-precision approximations. The same constraints apply.
<code>cifar_10_multiple</code>	<code>-DINT</code> <code>-DDOUBLE</code>	Same as <code>cifar_10</code> : the internal functions <code>lrn</code> and <code>softmax</code> require floating-point arithmetic and use single-precision approximations.
<code>correlation_2D</code>	<code>-DINT</code> <code>-DDOUBLE</code>	The algorithm computes mean values, normalization, and a square root. Using <code>-DINT</code> would eliminate all fractional results. The custom <code>bambu_sqrt</code> approximation uses single-precision literals, so <code>-DDOUBLE</code> would not accurately represent double-precision behavior.
<code>fft_2D_bench</code>	<code>-DINT</code> <code>-DDOUBLE</code>	FFT requires trigonometric twiddle factors and complex-valued rotations, which are incompatible with integer arithmetic. The <code>bambu_sin</code> and <code>bambu_cos</code> approximations use single-precision literals, making <code>-DDOUBLE</code> misleading.
<code>fft_bench</code>	<code>-DINT</code> <code>-DDOUBLE</code>	Same as <code>fft_2D_bench</code> : trigonometric twiddle factors preclude integer arithmetic, and the <code>bambu_sin</code> approximation uses single-precision literals.
<code>fft_window_bench</code>	<code>-DINT</code> <code>-DDOUBLE</code>	Same as <code>fft_bench</code> .
<code>softmax_bench</code>	<code>-DINT</code> <code>-DDOUBLE</code>	Softmax requires exponential evaluation and normalization by division, both of which produce fractional values. The Taylor series approximation uses single-precision literals, making <code>-DDOUBLE</code> and <code>-DINT</code> not representative.

Table 34: Benchmarks excluded from one or more datatype configurations, with justification.

References

- [1] PandA Project. *PandA Framework for High-Level Synthesis*. Available at: <https://panda.deib.polimi.it/>
- [2] Ferrandi, F. et al. *PandA-Bambu High-Level Synthesis Tool (GitHub Repository)*. Available at: <https://github.com/ferrandi/PandA-bambu>
- [3] AMD. *Vivado Design Suite*. Available at: <https://www.amd.com/es/products/software/adaptive-socs-and-fpgas/vivado.html>
- [4] OBPMARK. *GPU4S_Bench: GPU for Space Benchmark Suite*. Available at: https://github.com/OBPMARK/GPU4S_Bench